

目录

TensorRt&&部署问题	1
1. TensorRt	1
1.1 TensorRt 定义?	1
1.2 TensorRt 原理	2
低精度推理	2
使用 python 接口导入 tf 模型	4
1.3 安装 TensorRt	4
1.3.1 安装 TensorRT	5
1.3.2 安装 UFF, 支持 tensorflow 模型转化	5
1.3.3 安装 graphsurgeon, 支持自定义结构	5
2. Tensorflow 部署	5
签名的好处	6
3. Flask + Docker 简易部署	6
4. 基于 tflite 的 Android 部署	7

TensorRt&&部署问题

1.TensorRt

1.1 TensorRt 定义?

TensorFlow: pb, ckpt

pytorch: pt,.pth

keras: h5

显卡型号: tesla T4 | tesla V100

训练: V100、P40

推理: T4、P4

T4

整数运算能力 (INT8): 130 TOPS

GPU 显存: 16GB

显存带宽: 320GB/秒

定义: 推理加速 (基于 CUDA 和 cudnn 的进行高性能推理加速引擎); 用于从 ONNX 或 tensorflow 导入现有模型解析器, 是对训练好的模型进行优化。

当网络训练完后, 可以将 tf 训练模型文件直接丢进 tensorRT 中, 而不再需要依赖深度学习框架 (Caffe, TensorFlow 等)。

tensorRT 通过 组合层 和优化内核选择来优化网络, 从而改善延迟、吞吐量 (样本量/秒)、功效和内存消耗。

理解:

tensorRT 可以将 TensorFlow/pytorch 的网络模型解析, 然后与其中对应的层进行一一映射, 把其他框架的模型统一全部 转换到

tensorRT 中，然后在 tensorRT 中可以针对 NVIDIA 自家 GPU 实施优化策略，并进行部署加速。

TensorRT 对于网络结构进行了**重构和优化**，主要体现在：

1. tensorRT 通过解析网络模型将网络中无用的输出层消除以减小计算。
2. 对于网络结构的垂直整合，即将目前主流神经网络的 conv、BN、Relu 三个层融合为了一个层。
3. 对于网络的水平组合，水平组合是指将输入为相同张量和执行相同操作的层融合一起。
4. 对于 concat 层，将 concat 层的输入直接送入下面的操作中，不用单独进行 concat 后再输入计算，相当于减少了一次传输吞吐。

1.2 TensorRt 原理

低精度推理

TensorRT 支持 **FP16** 和 **INT8** 的计算，在训练的时候一般是使用 32 位或者 16 位数据，它在推理时可以**降低模型参数的位宽**来进行低精度推理，通过在**减少计算量**和**保持精度**之间达到一个理想的权衡，以达到**加速推断**的目的。模型在推理计算时，是调用 GPU 的 CUDA 核进行计算的。

P16 主要是 P100 和 V100 这两张卡支持；**INT8** 主要针对的是 P4 和 P40 这两张卡，P4 是专门针对线上做推断的小卡。

int8 量化算法

目标：将 fp32 的 CNN 转换为 int8 而不会造成明显的精度损失；

原因：int8 方法具有更高的吞吐量和更低的内存需求；

为什么要量化而不使用 int8 直接训练？

模型训练是需要反向传播和梯度下降的，训练时的超参一般都是浮点型，如学习率等，int8 类型无法进行训练。

samples 文件夹下有一个命令行包装工具，叫 **trtexec**，用来评测我们的网络模型，它有以下两个功能：

1. 在指定的网络模型中，对随机输入数据进行基准测试。eg：可利用 trtexc 工具来测试 UFF/ONNX net-model 在推理时的表现。
2. 对指定网络模型产生序列化引擎：使用完全**流水线异步**方式从多个线程运行具有多个执行上下文的引擎，以测试并行推理性能。

```
trtexec --deploy=/path/to/mnist.prototxt --
```

```
model=/path/to/mnist.caffemodel --output=prob --
```

```
batch=16 --saveEngine=mnist16.trt
```

定义： FP16(Full Precise Float 16)： (比 FP32)更省内存空间，更节约推理时间。

了解： **Tesla** 系列工作站端显卡： p4, p6, p40, p100 — 支持 Pascal 架构

do_inference 原理

host device 的方式转化将数据传输到 GPU, 然后异步执行做推断，推断的结果从 GPU 拿到 Host (即设备到主机)，接着将流进行同步。多个 input, output 结点用 do_inference_2 方法。

执行： **convert-to-uff** ./models/lenet5.pb --> 可生成 lenet.uff 文件

tensorflow: NWHC | tensorrt:NCWH (channel first)

uff-parser 解析 tensorflow 模型

先 **build_engine**, trt.UffParser() /caffe/ONNXparser;

再对图像进行处理，do_inference 进行异步方式推断

测试： python3 sample.py -d /home/featurize/work/TensorRT-6.0.1.5/data (需要 install pycuda)

序列化

序列化将引擎转换为一种格式，以便以后存储和使用以进行推理。要用于推理，只需反序列化引擎即可。

序列化和反序列化是可选的。由于从网络定义创建引擎可能非常耗时，因此每次应用程序重新运行时都可以通过序列化一次并在推理时对其进行反序列化来避免重建引擎。因此，在构建引擎之后，用户通常希望将其序列化以供以后使用。

可以序列化引擎，也可以直接使用引擎进行推理。在将模型用于推理之前，序列化和反序列化是一个可选步骤-如果需要，可以直接使用引擎对象进行推理。

序列化引擎不能跨平台或 TensorRT 版本移植。

序列化引擎步骤

将序列化引擎保存到文件中，并从文件中读回：

1. 序列化引擎并写入文件：

```
with open("sample.engine", "wb") as f:  
    f.write(engine.serialize())
```

2. 从文件读取引擎并反序列化：

```
with open("sample.engine", "rb") as f,  
    trt.Runtime(TRT_LOGGER) as runtime:  
    engine = runtime.deserialize_cuda_engine(f.read())
```

使用 python 接口导入 tf 模型

1. 导入 TensorRT : `import tensorrt as trt`
2. 创建冻结的 TensorFlow 模型
3. 使用 *UFF* 转换器将冻结的 Tensorflow 模型转换为 UFF 文件
`$ convert-to-uff frozen_inference_graph.pb`
4. 定义路径。更改以下路径以反映 Samples 中包含的模型的位置
`model_file = '/data/mnist/mnist.uff'`
5. 创建 builder , network 和 parser:

```
with builder= trt. Builder(TRT_LOGGER) as builder,  
    builder.create_network() as network,trt  
UffParser O) as parser:  
    parser.register_input("Placeholder", (1, 28, 28))  
    parser.register_output("fc2/Relu")  
    parser.parse(model_file, network)
```

builder: 搜索其 CUDA 内核目录以获得最快的可用实现, 因此必须使用相同的 CPU 来构建优化引擎将运行的 GPU.

builder 具有许多属性, 可以设置这些属性以控制网络应运行的精度, 以及自动调整参效。

两个重要的属性: 最大 batch size 和最大 workspace size.

最大 batch 大小: 指定 tensorRT 将优化的批量大小。在运行时, 可以选择较小的批量大小;

1.3 安装 TensorRt

下载地址: <https://developer.nvidia.com/zh-cn/tensorrt>

TensorRT 安装链接

#在 home 下新建文件夹, 命名为 tensorrt_tar, 然后将下载的压缩文件拷贝进来解压

```
tar xzvf TensorRT-6.0.1.5.Ubuntu-16.04.4.x86_64-  
gnu.cuda-9.0.cudnn7.3.tar
```

#解压得到 TensorRT-6.0.1.5 的文件夹, 将里边的 lib 绝对路径添加到环境变量中

```
export
```

```
LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/home/featurize  
/work/TensorRT-6.0.1.5/lib
```

1.3.1 安装 TensorRT

```
cd TensorRT-6.0.1.5/python
```

```
pip3 install tensorrt-6.0.1.5-py2.py3-none-any.whl
```

1.3.2 安装 UFF,支持 tensorflow 模型转化

```
cd TensorRT-6.0.1.5/uff
```

```
pip3 install uff-0.5.5-py2.py3-none-any.whl
```

1.3.3 安装 graphsurgeon, 支持自定义结构

```
cd TensorRT-6.0.1.5/graphsurgeon
```

```
pip install graphsurgeon-0.3.2-py2.py3-none-
```

```
any.whl
```

libcudart.so.10.0: cannot open shared object file 错误的解决方案都是查看 CUDA 的版本,

但是如果 CUDA 版本正确的时候还是报这个错误的话, 可以通过输入一下指令解决。

```
sudo ldconfig /usr/local/cuda-10.0/lib64
```

2.Tensorflow 部署

简单描述: 确定好输入和输出节点, 把模型导出成 SavedModel 格式, 然后用 TF-Serving 启动服务, 调用方发 http 请求或者 grpc 请求就可以拿到预测结果了。

保存权重文件:

1. ModelCheckpoint

2. model.save_weights()

keras 模型转化为 *SavedModel*:

```
tf.saved_model.save(model, '/keras_save_graph' )
```

查看模型文件命令:

```
saved_model_cli show --dir ./keras_save_graph --all
```

签名信息:

```
saved_model_cli show --dir ./keras_save_graph --tag-set
serve --signature_def serving_default
```

检查保存模型是否正常工作：（输入 2 个样本，28x28）

```
saved_model_cli run --dir ./keras_save_graph --tag-set
serve --signature_def serving_default
--input_exprs
```

```
'flatten_input=np.ones((2,28,28))'
```

还可以用程序进行验证：

```
loaded_saved_model =
tf.saved_model.load('./keras_save_graph')

print(list(loaded_saved_model.signatures.keys()))
inference = loaded_saved_model.signatures['serving_default']
```

使用 SaveModel 导出训练模型并添加签名

签名的好处

1. 可以限定函数的输入类型，以防止调用函数时调错，
2. 一个函数有了 input_signature 之后，在 tensorflow 里边才可以保存成 savedmodel。在保存成 savedmodel 的过程中，需要使用 get_concrete_function 函数把一个 tf.function 标注的普通的 python 函数变成带有图定义的函数。

3. Flask + Docker 简易部署

<https://www.cnblogs.com/ustcwx/p/12768463.html>

1. Saver 端：模型的离线训练与导出

使用 SavedModel，将训练好的模型导出为 pb 文件，检查保存的模型的 SignatureDef、Inputs、Outputs 等信息，命令
(saved_model_cli)

2. tf.server 进行模型加载与在线预测

基于 Docker 的方式搭建 tf-server，拉取 tf/serving 的镜像, 新建服务实例

3. Client 端 构建请求

Docker：容器，可以理解成一个“黑盒”。在项目变得庞大以后，往往我们会疲于管理整个项目的部署和维护。如果我们将整个项目

用一个“容器”装起来，那么我们仅仅只用维护一个配置文件告诉计算机每次部署要把什么东西装进“容器”，甚至借用一些工具把这个过程自动化，部署就会变得很方便。

部署环境：

Ubuntu18.04 显卡：Tesla 的 P40 搭建 Docker

Docker 封装 Flask 应用：

1. 创建一个 Dockerfile 文件，添加部署到服务器上的路径以及启动的 python 文件名和变量名。

```
FROM python:3.6
```

```
WORKDIR /Project/demo
```

```
COPY requirements.txt ./
```

```
RUN pip install -r requirements.txt -i
```

```
https://pypi.tuna.tsinghua.edu.cn/simple
```

```
CMD ["gunicorn", "start:app", "-c", "./gunicorn.conf.py"]
```

2. 开始构建 Docker 镜像，登陆远程服务器终端，把镜像 pull 下来。

```
sudo docker build -t 'testflask' sudo docker images
```

3. 部署到服务器上

```
sudo apt-get install docker.io
```

4. 运行 docker 镜像：

```
sudo docker run -d -p 80:80 --name test-flask-1
```

```
testflask
```

5. 更新项目的时候，维护好配置文件，build 一下，push 上去，在服务器 pull 下来，重新运行即可。

4. 基于 tflite 的 Android 部署

tf.lite.TFLiteConverter.from_saved_model()（推荐）：转换 SavedModel。

1. build.gradle：添加 tf-lite-gpu

2. model 放在 asset 文件下，tf 相关的文件放于 tflite 文件夹下

3. 设置 tfliteOption(nnapi、cpu、gpu)，构造函数

(loadModelFile)调用 tfliteModel, 初始化图像数据，分配内存；

4. 做预测的构造函数 recognizeImage, 先把 bitmap 转化为 ByteBuffer；

5. 接下来就是 runInference 操作 (tflite.run()), 得到所有的概率值。